
diskproxy

The diskproxy Manual

Virtual disks, NFS and WebDAV views that never write to the source

FirstPair contributors

2026-11-05

Version 1.0.0-a6ab31bf
Source commit a6ab31bf24f533497ca8a5a9ac9280ab416765f0

First Pair Press
firstpair.press

Contents

The diskproxy manual 2
What diskproxy is for 2
Installing 3
Everyday use 4
Configuration 5
Choosing a transport 6
Writable views 6
Workflow: camera-card ingest with Photo Mechanic Plus 7
Workflow: a photo library served from its previews 8
Writing a plugin 11
How it works 13
Limits 14
Troubleshooting 15
Development and verification 16
Command reference 16

The diskproxy manual

diskproxy presents a folder to macOS applications as a virtual disk, an NFS volume or a WebDAV share, without ever writing to the folder. A plugin decides what the application sees: which files are visible, which timestamps they carry, how each file's bytes are put together, and what happens to metadata the application writes back. Source bytes are read on demand, and disk metadata is built in memory, so nothing is copied and no kernel extension, FUSE driver or Python runtime is needed.

This manual is for people who want to use diskproxy, and for developers who want to adapt it. It covers installation, everyday commands, every configuration key, the three transports, writable views, two complete workflows, writing a plugin, the internal architecture, limits and troubleshooting. The README is the short version; the project story explains how the design came about.

What diskproxy is for

diskproxy solves one general problem: an application expects files to look a certain way, and you cannot or should not change the files themselves. Typical reasons are:

- the files are originals you want to protect, such as a camera card or a photo library;
- the application identifies files by properties that change for reasons unrelated to their content, such as filesystem timestamps;
- the useful version of a file is spread across two places, such as a small stand-in in one folder and its preview in another;
- the application writes metadata you want to keep, but not into the files.

Three plugins ship with diskproxy:

Plugin	What it presents	Typical use
`passthrough`	The folder as it is	A protected, read-only view of any folder
`pmp`	A camera card with stable modification times, optionally hiding photos already ingested	Photo Mechanic Plus incremental ingest across timezone changes
`previews`	Stand-in DNGs with a large preview JPEG embedded from another folder, and JPEGs with the camera, lens and XMP metadata of a matching DNG	Browsing and rating a photo library whose originals are offline

How it protects the source

Every source file is opened read-only, through a root directory descriptor, one path component at a time, refusing symlinks, `.` and `..`, and anything that is not a regular file or directory. Plugins never receive a writable descriptor. The HTTP transports listen only on localhost, behind a fresh random 192-bit token, and reject every request that would change something.

Writable views exist, but they are writable only inside a session: written bytes go to a shadow file or an overlay folder in the session directory, and `diskproxy stop` deletes them. Only what the plugin deliberately saves, such as the ratings captured by the previews plugin, outlives the session, and it is saved outside the source.

A running view is a snapshot of the folder's structure with live file bytes behind it. If a source file's size or modification time changes while it is served, reads of it fail rather than returning a mixture of old and new data. Stop and start again after an intended change.

Installing

diskproxy builds with Rust and Cargo. The launcher uses only tools included with macOS: zsh, `hdiutil`, `diskutil`, `plutil`, `mount_nfs`, `ps` and `nohup`. The server builds on any Unix, but automatic mounting is macOS only.

```
git clone https://github.com/firstpair/diskproxy.git
cd diskproxy
scripts/install.sh --config configs/passthrough.json
```

The installer builds a release binary and places the `diskproxy` launcher, the `diskproxy-server` binary and the older `pmp-card` compatibility commands in `~/bin`; make sure that folder is on your `PATH`. It seeds `~/.config/diskproxy/config.json` from the chosen example only if no default config exists yet, so reinstalling never overwrites your settings. The examples in `configs/` are:

Example	Plugin and mode
`passthrough.json`	A plain read-only disk of a folder
`pmp.json`	Camera-card ingest for Photo Mechanic Plus
`pmp-filtered.json`	The same, hiding photos already ingested
`previews.json`	A stand-in tree with embedded previews

To update, pull and run `scripts/install.sh` again. Stop a running session first, so the new binary is used from the next start.

Everyday use

Once the default config names a source folder and a plugin, these are the only commands you need:

```
diskproxy start      # or just: diskproxy
diskproxy status
diskproxy stop
```

`start` launches the server in the background, waits for it to scan the folder, mounts the result and prints where:

```
Ready: /Volumes/DiskProxy
Select this volume in your application (under Disks in PMP).
After use: diskproxy stop
```

Closing the Terminal window does not stop the view. `status` prints the server's process ID and the mounted volume. `stop` finishes any pending capture, unmounts, then stops the server and deletes the session folder.

Choosing a source or config for one session

A path after `start` overrides the configured source. `--config` selects another config file, before or after `start`, and `--set` overrides any single key:

```
diskproxy start /another/folder
diskproxy --config ~/views/library.json start
diskproxy start --config ~/views/library.json --set label=Library
```

Scanning large folders

Before it can serve a disk, the server reads the whole tree. For a camera card this takes a moment. For a large library the launcher shows progress on one line, updated in place every two seconds:

```
Scanning: listing folders: 3120 folders, 905000 files
Scanning: reading files: 523000 of 1256465 (41%)
```

The server logs a summary when it finishes, for example `scanned 1256465 files in 2413 folders in 59 s`. The launcher waits up to 15 minutes while the server is alive, and stops at once if the server exits with an error. Short scans print nothing. When output is not a terminal, each update is printed on its own line.

Measured on one library, a tree of 1.26 million files scanned in about a minute; the server then used about 1.2 GB of memory, with a 1.8 GB peak during startup. With camera metadata spliced into its 566,000 JPEGs (see the previews plugin below), the same tree scanned in 110 seconds and used 1.9 GB, with a 2.5 GB peak. Trees of up to 8 million entries are supported.

When the volume is busy

If an application still has files open, macOS refuses to detach the disk, and `stop` says so and leaves the server running, so the application can finish. Close the files and run `diskproxy stop` again. Before detaching, `stop` checks that the mounted volume is still the one it attached, by its volume UUID, so it never detaches some other disk.

The session folder

Each session lives in `~/Library/Caches/diskproxy/session` (set `DISKPROXY_STATE_DIR` to move it). It holds the server's startup JSON and error log, the process ID and start time, the attachment details and,

for writable views, the shadow file or overlay and the harvester's log. Only one managed session runs at a time; `start` refuses to start a second one.

Configuration

The default config is `~/.config/diskproxy/config.json`. A general example:

```
{
  "source": "/absolute/path/to/folder",
  "plugin": "passthrough",
  "mode": "disk",
  "label": "DiskProxy",
  "port": 0,
  "plugins": {}
}
```

Key	Meaning	Default
`source`	Folder to present; a path given to `start` overrides it	none, required
`plugin`	`passthrough`, `pmp` or `previews`	`passthrough`
`mode`	`disk`, `nfs` or `webdav`	`disk`
`label`	Volume name, up to 11 UTF-16 code units	plugin's default
`writable`	Disk mode only: mount through a shadow file and capture writes	`false`
`mountpoint`	NFS mode only: where to mount	`~/Volumes/LABEL`
`port`	Localhost port; `0` picks a free one	`0`
`plugins.<name>`	Settings for that plugin	`{}`

Paths in a config file are taken literally: write absolute paths, not ~. Unknown keys are errors, so a misspelling is reported rather than ignored.

Settings are applied in this order, later ones winning: the config file, then `--set KEY=VALUE`, then explicit flags such as `--label`. `--set` parses its value as JSON when it can, otherwise as a string, and reaches into plugin settings with dotted keys:

```
diskproxy start --set plugins.pmp.hide_ingested=true
diskproxy start --set 'plugins.previews.ratings="/absolute/path/to/ratings"'
```

The launcher reads `mode` from the config file to decide how to mount, so put `"mode": "nfs"` in the file rather than passing it with `--set`. The launcher refuses `--mode`, `--check` and `--register`; those belong to `diskproxy-server`.

Checking a config without serving

```
diskproxy-server --config ~/views/library.json --check
```

This validates the config and the plugin settings, prepares the plugin, prints its startup information as JSON and exits, without mounting or registering anything.

Choosing a transport

All three transports serve the same view: the same visible files, timestamps and composed bytes. They differ in what macOS sees.

	Disk	NFS	WebDAV
macOS sees	An exFAT disk attached by `hdiutil`	An NFSv3 volume mounted by `mount_nfs`	A WebDAV share
Launcher mounts it	Yes	Yes	No; use `diskproxy-server serve`
Counts as a disk	Yes, e.g. under Disks in Photo Mechanic	No	No
Writes	Optional, through a shadow file	Yes, into a session overlay	Rejected
Status	Ready	Experimental	Ready

Disk mode

The server generates an exFAT disk image in memory and serves it as one HTTP resource, `card.img`. macOS attaches it with `hdiutil` as a raw disk image and reads it with HTTP range requests. The boot region, allocation table and directory entries are generated; file contents are mapped to ranges of the source files and read on demand; unused space reads as zeros. The image never exists as a file.

Timestamps on the disk carry valid UTC offsets, so macOS does not reinterpret them in the current timezone. Names are written in composed Unicode (NFC), the form macOS's exFAT driver uses for lookups, while source files are still opened by their own spelling.

NFS mode

The server runs an NFSv3 server on localhost and the launcher mounts it with `mount_nfs`, which needs no administrator rights:

```
mount_nfs -o vers=3,tcp,port=P,mountport=P,nolocks,soft,intr,timeo=100,retrans=5 \
  127.0.0.1:/ ~/Volumes/LABEL
```

The server has no lock manager, so `nolocks` keeps file locking local to the Mac. Every file operation reaches the server by name, which is what makes NFS useful for capturing writes; see Writable views. NFS mode is experimental: see The NFS unmount hang.

WebDAV mode

WebDAV serves the tree as a read-only share. It is useful for tools and applications that work with folders rather than disks:

```
diskproxy-server serve /path/to/folder --plugin passthrough --mode webdav
```

The server prints a JSON line with a URL of the form `http://127.0.0.1:PORT/TOKEN/`; treat the URL as a password. It implements `OPTIONS`, `PROPFIND` with depth 0 or 1, `GET` and `HEAD` with single byte ranges, and rejects every other method with `403`. The transport guide lists the supported properties and limits.

Writable views

Some applications write metadata into the files they show: Photo Mechanic, for example, writes ratings, color labels and flags into JPEGs and DNGs. A read-only disk makes those writes fail. A writable view lets

them succeed without touching the source, and passes each written file to the plugin's `written` hook, which decides what to keep. Two ways are available.

	Shadow file	NFS
Enable with	`"writable": true` (disk mode)	`"mode": "nfs"`
Status	Ready	Experimental
Mounted at	`/Volumes/LABEL`	`~/Volumes/LABEL`, or `mountpoint`
Written bytes go to	`disk.shadow` in the session folder	`overlay/` in the session folder
The plugin sees a write	Within about a minute, and on stop	About two seconds after the last write, and on stop

Either way, the application reads its own changes back during the session, and `diskproxy stop` deletes the written bytes after the final capture.

Shadow-file mode

The disk is attached with `hdiutil attach -shadow`. macOS then treats it as writable, but every changed block goes to the shadow file, and the served image stays read-only. Because the disk delivers blocks rather than files, diskproxy cannot see writes directly. Instead, a harvester process follows filesystem events on the mounted volume, and every minute passes the changed files to the plugin. It reads each file from the mounted volume, so it sees the application's version.

`diskproxy stop` asks the harvester for a final pass and waits up to two minutes for it before detaching. If the harvester is not running, `stop` first walks the whole volume once for files modified since mounting, so no change is lost. If either step fails, the disk stays mounted and `stop` points at the harvester log.

The harvester can also be run by hand, for example after a crash left a writable volume mounted:

```
diskproxy-server harvest --config view.json --volume /Volumes/VIEW \
  --since EPOCH --once
```

`--since` walks the volume for files modified at or after that Unix time; `--once` returns after that walk instead of watching. Without `--once` it keeps watching, every `--interval` seconds (default 60), until the `--stop-file` appears. Hidden housekeeping folders such as `.fseventsd` are skipped.

NFS mode

The first write to a served file copies its presented bytes into the overlay folder; from then on the client reads and writes that copy. New files, new folders, renames and deletions are recorded in the overlay as well, so an application that saves through a temporary file and a rename works normally. Two seconds after the last write to a file, the file is passed to the plugin. When the server receives `SIGTERM`, which is how `diskproxy stop` ends it, it passes everything still pending before exiting.

Workflow: camera-card ingest with Photo Mechanic Plus

This is the workflow diskproxy was first written for. The PMP guide is the complete reference.

The problem

Photo Mechanic Plus records each ingested file's path, size and modification time, and its incremental ingest copies only files it has not seen. On the exFAT cards examined, the camera wrote modification times without

a valid timezone offset, so macOS interpreted them in whatever timezone the Mac was in when the card was mounted. After travelling, the same photo had a different modification time, Photo Mechanic thought it was new, and it copied the whole card again.

The fix

The `pmp` plugin presents the card as a virtual disk with a fixed timestamp convention. For a new card the convention is UTC; a card with existing history keeps the convention its history was made in, for example Pacific time. The plugin works out the offset the card was mounted in from up to three sampled photos, recorded in a small profile on the card when it is registered. The photos themselves, and their EXIF capture times, are never changed.

Setting up

```
scripts/install.sh --config configs/pmp.json
```

The example config expects the card at `/Volumes/LEICA M`; edit `source` for another card name, or pass the path to `start`.

Registering a new card

Finish any EXIF corrections first, then, with the card mounted in the timezone the Mac is currently set to:

```
diskproxy-server
```

With no command, `diskproxy-server` registers the card: it writes `.pmingest.dat` if it is absent and the profile `.pmp-ingest-proxy.json` to the card, and exits. If you run `diskproxy start` on an unregistered card, it simply prints `Run diskproxy-server.` If the Mac's timezone changed while the card stayed mounted, remount it first, or give the offset it was mounted in:

```
diskproxy-server --source-offset=+09:00
```

Running the command again on a registered card reuses its profile unchanged. `--register` deliberately refreshes the samples after reviewed edits.

Ingesting

```
diskproxy start
# In Photo Mechanic Plus, ingest from the PMP Pacific volume under Disks.
diskproxy stop
```

Then eject the card. The volume is called `PMP Pacific` for compatibility, even for UTC cards; the label does not set the convention.

To hide photos already ingested, rather than relying on Photo Mechanic's incremental history:

```
diskproxy start --set plugins.pmp.hide_ingested=true
```

The history is read once at start, so restart the view to hide newly ingested files.

Workflow: a photo library served from its previews

The `previews` plugin builds a browsable library out of files that already exist. Its guide is the complete reference; this section walks through a whole setup.

The idea

Suppose your originals are offline, but you have two smaller things for every photo:

- a stand-in: a small DNG such as a Lightroom Smart Preview, which holds the photo's metadata and a reduced raw image, but embeds only a tiny thumbnail;
- a preview JPEG at a useful size, for example from an image browser's preview cache, under a different name in a different folder.

An application that shows embedded previews, such as Photo Mechanic or Quick Look, shows only the thumbnail in the stand-in. The previews plugin serves each stand-in with its preview JPEG embedded, without writing or copying either file.

Building the stand-in tree

Arrange the stand-ins under the folder layout you want to browse, usually the originals' layout. On APFS, use clones (`cp -c`) rather than copies: a clone shares its blocks with the original, so even a million stand-ins cost almost no disk space, and writing to a clone never changes the file it came from.

Then write a map file, one line per file to change, as a path relative to the tree followed by tab-separated fields:

```
2024/trip/IMG_0001.DNG  preview=a1/IMG_0001.jpg
2024/trip/IMG_0002.JPG  orientation=6   datetime=2024:10:02 04:45:42
2024/trip/IMG_0003.JPG  metadata=0/0A1B/0A1B2C3D.dng
```

Key	For	Value
`preview`	DNG, TIFF	The preview JPEG, relative to `preview_root`
`preview_size`	DNG, TIFF	Optional: the JPEG's size in bytes
`preview_mtime`	DNG, TIFF	Optional: its modification time, `SECONDS.NANOSECONDS`
`preview_dims`	DNG, TIFF	Optional: its pixel size, `WIDTHxHEIGHT`
`metadata`	JPEG	A DNG whose EXIF and XMP the JPEG is served with, relative to `metadata_root`
`orientation`	JPEG	EXIF orientation, 1 to 8
`datetime`	JPEG	Capture time, `YYYY:MM:DD HH:MM:SS`

With all three optional preview fields, the server builds the disk without opening a million preview JPEGs, which makes startup much faster. They are still checked against the real file on every read.

What gets served

For a DNG or TIFF with `preview=`, every original byte stays at its original offset. The plugin appends a copy of the first image directory (IFD0) after the original data, adds one entry pointing at a new reduced-resolution image (a SubIFD), and appends the JPEG. Only the 8-byte TIFF header is changed, to point at the copied directory. Applications that show embedded previews then find the large JPEG; raw-capable applications still read the original image data. An existing large preview is kept and stays first.

For a JPEG without EXIF, `metadata=` adds the EXIF and XMP of a DNG, typically the photo's Lightroom Smart Preview, so the JPEG shows its camera, lens, exposure, serial number and GPS position. The plugin rebuilds the EXIF in a new segment, leaving out the DNG's image-structure tags and maker note and adding

the JPEG's own pixel size. It reads the XMP segment's contents from the DNG in place on every read, so they take no memory. The rebuilt EXIF, about 1.3 KB per JPEG, is kept in memory with the rest of the layout. `orientation=` and `datetime=` set Orientation and DateTimeOriginal in that segment, overriding the DNG, so portraits display upright and photos sort by capture time; they also work on their own. JPEGs that already have EXIF are served unchanged.

Files not in the map are served unchanged. A mapped file that cannot be composed, because its JPEG is missing or the DNG has an unexpected structure, is served unchanged with a warning in the server log, rather than failing the disk.

The config

```
{
  "source": "/absolute/path/to/stand-in-tree",
  "plugin": "previews",
  "label": "Library",
  "writable": true,
  "plugins": {
    "previews": {
      "map": "/absolute/path/to/previews.tsv",
      "preview_root": "/absolute/path/to/preview-jpegs",
      "metadata_root": "/absolute/path/to/Catalog Smart Previews.lrdata",
      "ratings": "/absolute/path/to/ratings"
    }
  }
}
```

`metadata_root` is needed only when the map has `metadata=` fields. Leave out `writable` and `ratings` for a purely read-only library. Start it with `diskproxy --config library.json start`. Check one file through the mounted volume:

```
exiftool -b -PreviewImage /Volumes/Library/2024/trip/IMG_0001.DNG | wc -c
```

The result should be the size of its preview JPEG.

Capturing ratings

With `ratings` set and a writable view, the plugin reads each written file's XMP metadata and keeps exactly four properties:

Property	Meaning
`xmp:Rating`	Star rating
`xmp:Label`	Color label name
`photomechanic:ColorClass`	Photo Mechanic's color class
`photomechanic:Tagged`	Photo Mechanic's tag (flag)

Everything else an application writes is ignored. That is deliberate. Photo Mechanic, for example, sets creation dates from a file's modification time when a JPEG has none, and copies the stand-in's metadata into its XMP. Keeping its whole XMP block would carry those invented values back towards your originals.

The metadata is found in a JPEG's XMP segment, a TIFF or DNG's XMP tag, or an `.xmp`/`.XMP` sidecar. Each photo gets a minimal sidecar in the ratings folder, mirroring the tree:

Written file	Sidecar
`X.DNG`, or its sidecar `X.XMP`	`X.xmp`
`X.JPG`	`X.JPG.xmp`, so a JPEG never collides with a DNG of the same name
A stand-in served as `X.JPG.dng` or `X.RW2.dng`	Under the original's name: `X.JPG.xmp`, `X.xmp`

A sidecar is rewritten only when a value changed. Every change is appended to `JOURNAL.jsonl` in the ratings folder, one JSON object per line:

```
{"time_unix":1793718040.97,"written":"2015/February/IMG_4193.dng",
 "sidecar":"2015/February/IMG_4193.xmp",
 "fields":
{"xmp:Rating":"2","xmp:Label":"Red","photomechanic:ColorClass":"1","photomechanic:Tagged":"False"},
 "previous":{}}
```

`written` is the file the application wrote, `sidecar` the file updated in the ratings folder, and `previous` the values before this change. The ratings folder must be outside the served tree; the plugin refuses to start otherwise.

These sidecars use the Adobe naming convention, so a DNG's sidecar can later sit beside its original, where Lightroom or another application can read it.

Writing a plugin

A plugin is a Rust crate that implements the `Plugin` trait from `diskproxy-core` and is linked into the application at build time. There is no dynamic loading: adding a plugin means adding a crate and registering it in `src/main.rs`. The plugin guide gives every step, and the hide example is a complete, compiled reference with a patch that registers it.

The trait

```rust
pub trait Plugin: Send + Sync {
    fn identity(&self) -> Option<String> { None }
    fn visible(&self, _parts: &[String], _metadata: &Metadata) -> Result<bool>
{ Ok(true) }
    fn mtime(&self, _parts: &[String], metadata: &Metadata) -> Result<i64>
{ Ok(metadata.mtime()) }
    fn compose(&self, _parts: &[String], _file: &File) -> Result<Option<Vec<Piece>>>
{ Ok(None) }
    fn written(&self, _parts: &[String], _file: &File) -> Result<()> { Ok(()) }
}
```

Every method has a default, so a plugin overrides only what it needs. `parts` are the path components relative to the source root.

Hook	Return	Meaning
`identity`	Stable string or `None`	Seeds the disk's volume serial; `None` derives it from the source
`visible`	`true` or `false`	Shows or hides an entry, on direct opens as well as in listings
`mtime`	Unix seconds	The modification time to present; the source keeps its own
`compose`	Pieces or `None`	Presents a file as a sequence of pieces; `None` serves it unchanged
`written`	`()`	A client wrote this file on a writable view; `file` holds its new bytes

An error aborts the operation; it is never a request to fall back to the unmodified entry. Keep the hooks deterministic for a session, because the disk is built from one snapshot of the tree. Plugins are shared across threads.

Composing files

```
pub enum Piece {
    Bytes(Vec<u8>),
    Own { offset: u64, length: u64 },
    Other {
        source: Arc<Source>,
        path: Arc<str>,
        offset: u64,
        length: u64,
        stamp: Option<Stamp>,
    },
}
```

`Bytes` are generated by the plugin, `Own` is a range of the file being presented, and `Other` is a range of a file under another root. Open that root with `Source::new`, so it gets the same read-only, no-symlink rules. The core checks every range against the file's size, records each file's size and modification time, and refuses reads once either changes. When the plugin already knows another file's size and modification time, it can pass them as `stamp`, so the scan doesn't open that file; the stamp is still checked on every read. Composed files are served identically by all three transports.

Receiving writes

`written` is called with a read-only handle to the new bytes: from the NFS overlay, or from the mounted volume for a harvested shadow-file view. The source is never written, and the handle cannot write to it. A plugin that saves something should save it outside the source, as the previews plugin's ratings folder must be, and should treat the call as untrusted input: temporary files, partial saves and files the application writes for its own purposes all arrive here.

Registering and testing

To register a plugin, add its crate to the workspace and the application's dependencies, add its name to `PluginName` in `src/main.rs`, validate its `plugins.<name>` settings before any setup I/O, and add a construc-

tion arm that returns the source, the plugin and its startup information. Test with synthetic folders only, through both direct opens and listings, and confirm that the source files are unchanged afterwards.

A plugin is trusted code. The trait is a policy boundary, not a sandbox: a plugin can call any API Rust allows, so keep its own writes small, explicit and documented.

How it works

Components

Component	Location	Responsibility
Core	`crates/core`	Read-only source access, the view, the exFAT builder, HTTP, WebDAV, NFS and harvesting; nothing application-specific
PMP plugin	`plugins/pmp`	Camera layout, card profiles, timestamp conventions and history filtering
Previews plugin	`plugins/previews`	The preview map, DNG and JPEG composition, and ratings capture
Application	`src/main.rs`	Config loading, overrides, plugin selection, and the `harvest` and `unmount` commands
Launcher	`bin/diskproxy`	The background process, mounting, harvesting and the start/status/stop lifecycle

Sources and views

A `Source` opens its root directory once. Every lookup walks from that descriptor one component at a time with `openat` and `O_NOFOLLOW`, rejecting empty components, `.`, `..`, embedded slashes and NUL, and returning only read-only regular files and directories. A `View` combines a source, a plugin and a volume label, and every transport serves through it.

Building the disk

The scan runs in two phases. A serial walk lists every folder and adds a placeholder for each file. Then a parallel phase opens and lays out the files, one directory per task, each file opened relative to its already-open folder, and calls the plugin's `compose` for each. A reporter thread prints progress every two seconds on long scans.

The layout keeps each name once, with a link to its parent folder, and rebuilds full paths only when a file is read; each file's pieces become extents in the disk's map. After building, the server returns freed startup memory to macOS, so its resident size reflects what it keeps.

A disk read finds the extents covering the requested range: generated metadata, zeros, or ranges of source files. Before reading a source file, it checks that file's size and modification time against what was recorded at scan time.

The NFS server

The NFS transport is built on the `nfsserve` crate and runs on tokio. It presents the same view by path. Its overlay keeps client-written files, client-created folders and the paths the client removed or renamed away. Presented metadata is computed on first use. A settle timer passes each written file to the plugin once it has had no writes for two seconds, and a `SIGTERM` handler flushes the rest.

The launcher

`bin/diskproxy` starts `diskproxy-server serve` in the background with its output in the session folder, polls the startup JSON for the disk URL or NFS port, shows scan progress, and mounts. For a writable disk it also starts the harvester and records its arguments, start time and the mount time, so `stop` can finish or replace it. `stop` reverses each step and checks identities on the way, so it never detaches a different disk or stops a different process.

Limits

- **Disk mode:** names must be UTF-8 and valid for exFAT, and timestamps must fall between 1980 and 2107. The case-folding table covers ASCII letters only. Unsupported names and times fail; they are not rewritten.
- **Previews plugin:** classic TIFF only (not BigTIFF), files under 4 GiB. A preview is added, never replaced. JPEGs that already contain EXIF are served unchanged. A metadata DNG's XMP is served as it is, so its `dc:format` still says `image/dng`.
- **One session at a time** through the launcher.
- **A stable source:** files changed while served fail to read; restart after intended changes. A plugin's prepared state, such as the PMP history snapshot or the previews map, is fixed for the session.
- **Writable views are per session.** Written bytes are discarded on stop; only what a plugin saves is kept.
- **NFS mode is experimental.**
- **Local use only.** There is no TLS, no user authentication and no sharing across a network; the HTTP URL is a localhost capability.

Troubleshooting

Symptom	Cause and fix
`Run diskproxy-server.`	The camera card is not registered; see Registering a new card
`A managed session already exists.`	A view is already running; use `diskproxy status` or `diskproxy stop`
`Detach failed. Server left running...`	An application still has files open; close them and run `diskproxy stop` again
`Mounted volume identity changed. Server left running.`	The mounted disk is not the one diskproxy attached; check it before doing anything else
`Server did not provide a disk URL within 15 minutes.`	The scan did not finish; the server's errors are printed above the message
A read fails with an I/O error	A source or preview file changed while served; stop and start again
`Ratings harvest still running ...`	The final capture took over two minutes; check the harvester log in the session folder and retry
`Ratings check failed; disk left mounted.`	The replacement walk failed; see the harvester log, fix the cause and retry
`Mount point ... is not empty.`	NFS mode mounts only on an empty folder; empty it or set `mountpoint`
A DNG shows only a small thumbnail	Its map entry is missing, or its preview could not be composed; the server log has a warning

The NFS unmount hang

During early development on macOS 26, unmounting an NFS view hung inside the kernel's NFS client three times, for between one minute and about half an hour. The volume disappeared at once, but the unmounting process sat in an uninterruptible wait with no NFS traffic, and later NFS mounts queued behind it. Since then the current server has passed 142 consecutive mount/write/unmount cycles without a hang. They were 82 direct cycles, covering both unmount methods, delayed and immediate unmounts and temporary-file-and-rename rewrites, plus 60 cycles through `diskproxy start` and `stop`. Unmounts took about 0.1 seconds and the slowest stop took 4.6 seconds. The earlier hangs may have come from the first server version and a kernel client it left degraded; the cause is not proven.

If it happens:

- leave the stuck process alone; it clears on its own, and killing it does not help;
- don't start another NFS view until it clears, because new mounts queue behind it;
- use a shadow-file view in the meantime; it does not use NFS.

The launcher unmounts with the `unmount(2)` system call, through `diskproxy-server unmount [--force] PATH`. Avoid `umount` and `diskutil unmount` for NFS views: they wait minutes afterwards for an rpcbind registration this server does not make.

Development and verification

```
cargo test --workspace --locked
cargo clippy --workspace --all-targets --locked -- -D warnings
uv run --python 3.13 python -m unittest discover -s tests -p 'test_*.py'
uv run --python 3.13 python scripts/check-plugin-example.py
```

The Rust tests use synthetic sources. They cover path and range validation, read-only access and symlink rejection, exFAT timestamps and checksums, NFC names, composed reads across every piece boundary and change detection on their pieces, the DNG preview patch, JPEG EXIF injection and metadata splicing from a DNG, known preview stamps, map validation, the PMP timezone conventions, ratings capture from JPEG, DNG and sidecar XMP, sidecar naming and journaling, and harvesting by walk and by filesystem events. Integration tests run the server binary for config precedence, unknown settings and HTTP behavior. The Python tests drive the launcher against a mocked server and `hdiutil`: clean stops, busy detaches, identity mismatches and registration advice. The plugin check registers the hide example in a copy of the repository and serves it.

Beyond the tests, the workflows in this manual were exercised on real data. The PMP plugin resumed an interrupted card at its first new frame, and a new card was registered and ingested with the UTC convention. A previews library of 1.26 million stand-ins served its embedded previews byte for byte, and captured ratings written by Photo Mechanic and exiftool.

Never use real photos, catalogs or application databases as test fixtures.

Command reference

`diskproxy`

Command	Effect
`diskproxy [--config FILE] [start] [SOURCE] [options]`	Start a view in the background and mount it
`diskproxy status`	Show the running server and volume
`diskproxy stop`	Capture pending writes, unmount, stop the server and delete the session
`diskproxy --help`	Short usage

Options after `start` are passed to the server, for example `--set KEY=VALUE` and `--label NAME`.

diskproxy-server

Command	Effect
`diskproxy-server`	Register a camera card with the default PMP config, then exit
`diskproxy-server register [options]`	The same, explicitly
`diskproxy-server serve [SOURCE] [options]`	Serve in the foreground; prints startup JSON
`diskproxy-server --config FILE --check`	Validate and print the startup JSON without serving
`diskproxy-server harvest --config FILE --volume PATH [--since EPOCH] [--once] [--interval S] [--stop-file PATH]`	Capture writes from a mounted writable volume
`diskproxy-server unmount [--force] PATH`	Unmount with `unmount(2)`

Option	Meaning
`--config FILE`	Config file; `DISKPROXY_CONFIG` sets the default
`--plugin NAME`, `--mode MODE`, `--label NAME`, `--port N`	Override those keys
`--set KEY=VALUE`	Override any key, including plugin settings
`--overlay DIR`	NFS: the overlay folder (the launcher sets it)
`--source-offset`, `--target-offset`, `--target-zone`, `--history`, `--profiles-dir`	PMP plugin settings
`--register`	PMP: refresh the card's samples deliberately